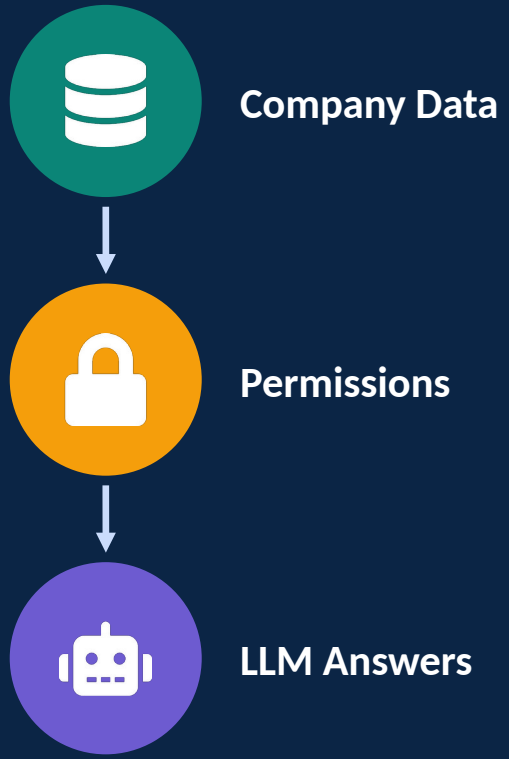
AI PROJECT PROPOSAL

Permission-Aware Enterprise RAG

“Second Brain”: AI search that knows the whole company, but shows each person only what they are allowed to see.



Full development + production deployment | RAG · Security · MLOps

The Problem



Scattered Data

Drive, Slack, Jira, GitHub, Notion: employees search 5+ tools for a single answer.



Hallucination

Plain LLMs never saw private company data, so they guess and sound confident.



Security Leak

A chatbot over all documents lets an intern read the CEO's salary sheet.

Goal: one chat that answers with citations, using only data the user is allowed to see.

The Solution in Action

Question: “What was our Q3 revenue?”



Rahul · Intern

Groups: all-employees

No accessible document found for this question.



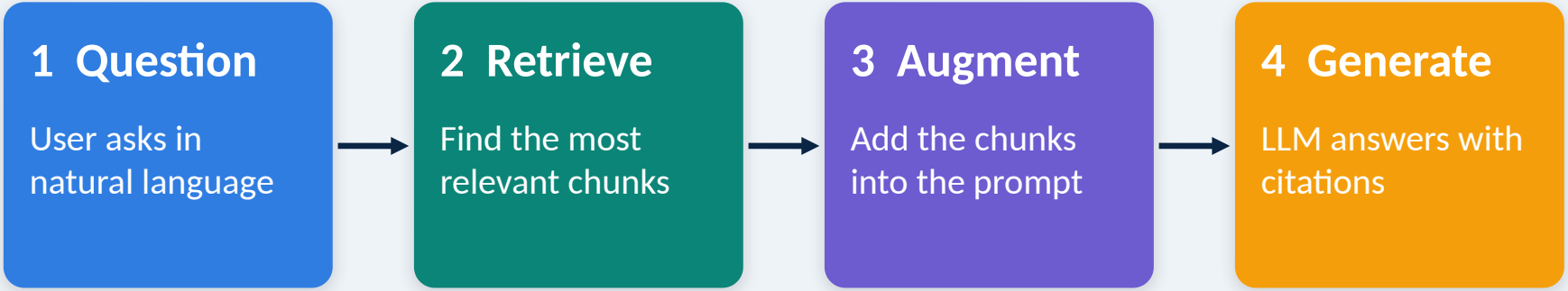
Priya · CFO

Groups: finance, leadership

Q3 revenue answer with citation: Finance_Q3.xlsx [1]

Same system, same question, different answers: permissions decide.

How RAG Works



Why use RAG?

Accurate

Answers come from real documents

Always fresh

No retraining when data changes

Trusted

Every claim has a citation

Core Concepts Behind the System

Embeddings

Text turned into number vectors so similar meanings sit close together.

Vector Database

Qdrant stores millions of vectors and finds the nearest fast (HNSW).

Chunking

Split docs into 300-800 token pieces by headings, tables and code.

Hybrid Search

Meaning search + BM25 keywords, merged with Reciprocal Rank Fusion.

Reranking

A cross-encoder re-scores the top 50 results and keeps the best 6.

Evaluation

RAGAS measures context recall, faithfulness and answer relevancy.

Permission-Aware Retrieval

Post-filter: WRONG

1 Search all chunks, take top 10

↓

2 Remove restricted chunks

↓

3 Only 2 results left; existence may leak

Pre-filter: OUR DESIGN

1 Apply user's ACL filter inside the DB

↓

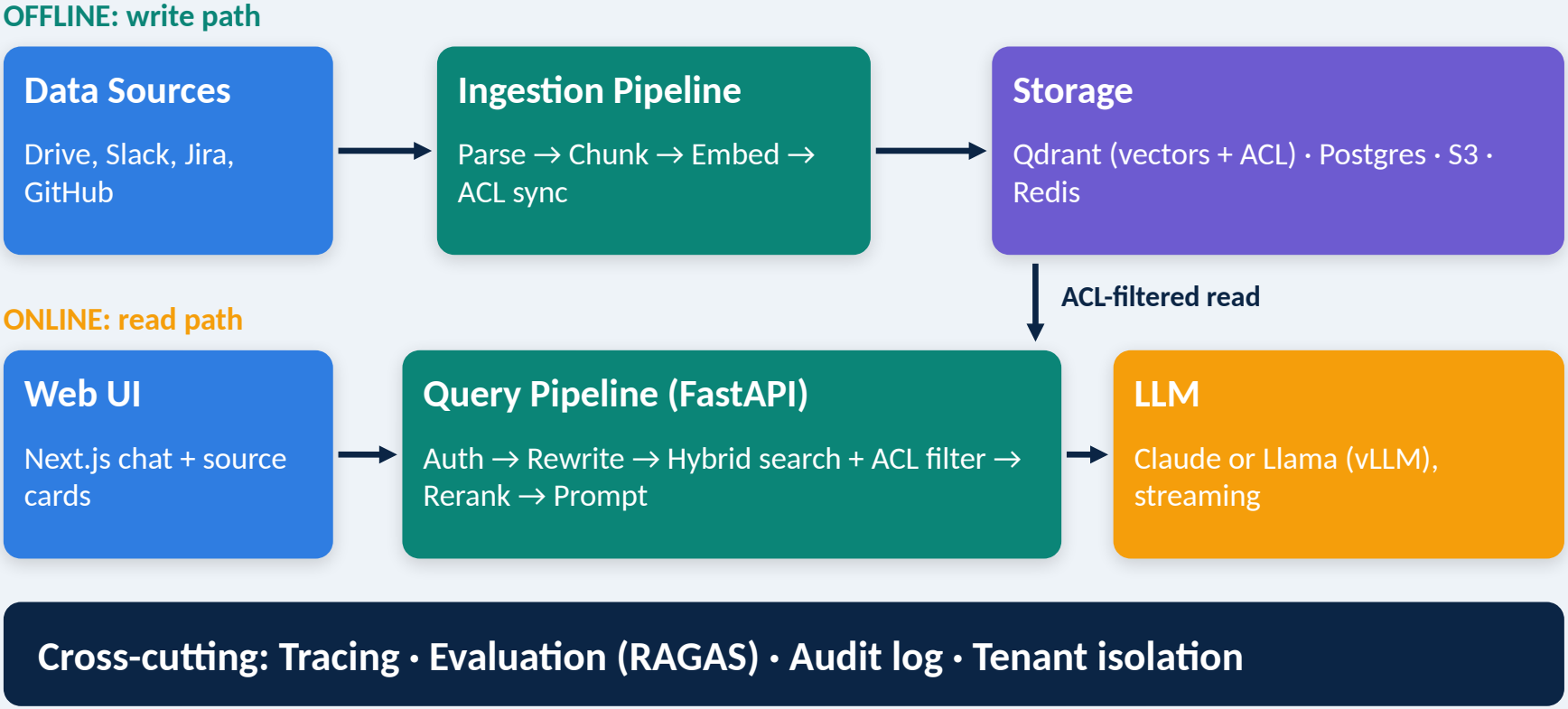
2 Search only chunks the user may see

↓

3 Full top results, nothing leaks

Golden rule: security is enforced in code and the database, never left to the LLM.

System Architecture



Life of One Query

Example: “What is the payment service rollback procedure?”

1. User sends question with login token
2. Verify identity, load user's groups
3. Rewrite query using chat history
4. **Hybrid search with ACL pre-filter**
5. Rerank top 50 down to best 6
6. Build prompt: rules + chunks + question
7. LLM streams the answer
8. Attach citations [1] [2]
9. Show source cards in the UI
10. Log trace, audit and feedback

10 Modules and Tech Stack



01

Connectors

Drive, GitHub, Slack, Jira



02

Ingestion

Celery, Docling, embeddings



03

Identity & ACL

OIDC SSO, Postgres, Redis



04

Retrieval

Qdrant, BM25, reranker



05

Generation

Claude API, vLLM, guardrails



06

Backend API

FastAPI, SSE streaming



07

Frontend

Next.js, Tailwind, shadcn



08

Evaluation

RAGAS, pytest, golden set



09

Observability

Langfuse, Prometheus, Grafana



10

Deployment

Docker, K8s, Terraform

Roadmap, Quality and Outcome

Build roadmap (4-6 weeks)

Stage 1
Setup
Docker, Postgres, Qdrant, Redis, FastAPI skeleton

Stage 2
Basic RAG
Upload → parse → chunk → embed → search

Stage 3
Quality RAG
Hybrid search, reranker, citations, streaming chat

Stage 4
Permissions
Users, groups, ACL pre-filter, security tests

Stage 5
Production
Connectors, UI, evals, K8s deploy, monitoring

Target results (to be measured)

0
permission leaks in security tests

≥ 0.9
faithfulness score (RAGAS)

< 3 s
p95 time to first answer

10M+
chunks supported at scale